System and Software User Manual

STAR: Spatial Topography Accessibility Robot

Operator's Guide for Daily Use, Setup, and Maintenance

Team Locked Inc

Cesar Magana	Software
Brighton Sikarskie	Software, Embedded Development, Schematic Design
Jeremie Hockey	PCB Design and Layout
Michael Norton	Mechanical Design
Shawn Ciaciura	Schematic Review
Jared Bartz	Schematic Review

Course

ESET 420 – Senior Design II – Dr. Lee Hudson

Institution

Texas A&M University
Department of Electronic Systems Engineering Technology (ESET)
ESET Senior Capstone, Spring 2026

Advisors and Sponsors

Dr. Logan Porter – Sponsor and Technical Advisor

April 28, 2026

Version 1.0 – Capstone Submission

Contents

List of Figures

List of Tables

1 Scope

1.1 Purpose

This manual is the operator-facing companion to the STAR Software Description Document (SDD). The SDD explains HOW the STAR software is designed and built; this manual explains HOW TO USE it. If a procedure here is insufficient, consult the SDD for the underlying design, or consult the source repository for authoritative detail.

1.2 Audience

The intended readers are:

- Capstone reviewers validating that STAR can be operated by a non-developer through documented procedures.
- Follow-on student teams inheriting the robot and needing to bring it up on new hardware or at a new site.
- ADA facility auditors or building managers piloting STAR for a real compliance run.

Knowledge of Linux shells, SSH, and browser-based dashboards is assumed; knowledge of ROS2 internals, C firmware, or protobuf is not.

1.3 Out of Scope

This manual does not cover software architecture, algorithm derivations, data formats, testing strategy, or security design decisions. Those live in the STAR SDD. Mechanical assembly, PCB schematics, and BOM are covered in their own capstone deliverables.

1.4 Status Label Legend

Throughout this document, three status labels identify how complete each subsystem or feature is on the delivered system. Every body occurrence of `[IMPLEMENTED]` / `[STRETCH]` / `[ARCHITECTED]` elsewhere in this manual is a hyperlink back to this legend.

- [**IMPLEMENTED**] – complete and exercised in daily use.
- [**STRETCH**] – partial implementation with scaffolding; fit for demonstration but not yet complete.
- [**ARCHITECTED**] – designed and specified but not coded.

2 Referenced Documents

- STAR Software Description Document (companion capstone deliverable, `final_docs/software_description_`
- `docs/ARCHITECTURE.md` – end-to-end operations stack topology (Pi5, Tailscale, k3s, Cloudflare).
- `infra/README.md` – Pi5-side system files, Caddy configuration, systemd units, and fresh-Pi5 install procedure.
- `docs/PI_DEPLOYMENT.md` – ROS2 Jazzy workstation and Pi5 provisioning.
- `scripts/flash-rx72n.sh` – RX72N firmware flashing helper.

- `monitoring/grafana-star-dashboard.json` – Grafana dashboard definition used by the operator cockpit.
- `star-ros2/src/star_simple_bridge/` – MVP ROS2 bridge that serves Prometheus metrics and map images.
- `star-ros2/src/star_bringup/foxglove/star-default.json` – default Foxglove / Lichtblick layout.
- U.S. Department of Justice, *2010 ADA Standards for Accessible Design*, Section 405.2 (Ramp Running Slope). https://www.access-board.gov/ada/chapter/ch04/
- Open Navigation LLC, Nav2 navigation framework documentation. https://docs.nav2.org/
- Lichtblick, `lichtblick-suite/lichtblick` static web bundle (Foxglove Studio fork). https://github.com/lichtblick-suite/lichtblick
- Open Robotics, ROS 2 Jazzy Jalisco (LTS, May 2024 – May 2029, Ubuntu 24.04). https://docs.ros.org/en/jazzy/

3 Software Summary

3.1 What STAR Does

STAR is an autonomous indoor mobile robot that performs ADA Title III compliance audits. The operator drives or dispatches the robot into a building; the robot maps the space, measures geometric features (ramp slope, path width, trip hazards), and produces a structured compliance report. All of the control software runs on a Raspberry Pi 5 on the robot; a second microcontroller (Renesas RX72N) performs real-time motor control. The operator interacts with STAR through web-based dashboards – primarily Grafana for control and Lichtblick (or Foxglove) for 3D visualization.

3.2 Operator-Visible Software Inventory

Table 1 summarizes what the operator actually sees and uses. Everything underneath is described in the SDD.

Table 1: Operator-visible software inventory.

Component	Where	What it does for the operator
Grafana dashboard	k3s cluster / browser	Primary cockpit. Live telemetry, autonomy toggle, e-stop, speed, SLAM controls. [IMPLEMENTED]
Lichtblick web app	k3s cluster / browser	3D visualization of LiDAR, map, odometry, and TF tree. Foxglove-compatible. [IMPLEMENTED]
Foxglove Studio desktop	Laptop (tailnet)	Power-user alternative to Lichtblick, same layout file. [IMPLEMENTED]
`star_cockpit_api`	Pi5 `:9102`	HTTP API backing the Grafana cockpit toggles. [IMPLEMENTED]
`star_simple_bridge`	Pi5 `:9101`	ROS2 serial bridge to RX72N, Prometheus metrics, map image server. [IMPLEMENTED]
Caddy (Pi5-side)	Pi5 `:443`	Local-CA TLS front for `star.local`. [IMPLEMENTED]
`star-slam-mvp.service`	Pi5 systemd	Autostarts the SLAM MVP node set on boot; safe MANUAL state. [IMPLEMENTED]
RX72N firmware	On-board MCU	Closed-loop motor control, encoder and IMU decoding. [IMPLEMENTED]
`flash-rx72n.sh`	Workstation	One-shot firmware flashing helper. [IMPLEMENTED]

3.3 Operator Connection Topology

Figure 1 summarizes how an operator in a browser reaches the robot. The Pi5 is on a Tailscale mesh (IP `100.64.0.6`); a k3s homelab cluster hosts Grafana and Lichtblick; a public Caddy instance on the homelab fronts both for Cloudflare TLS.

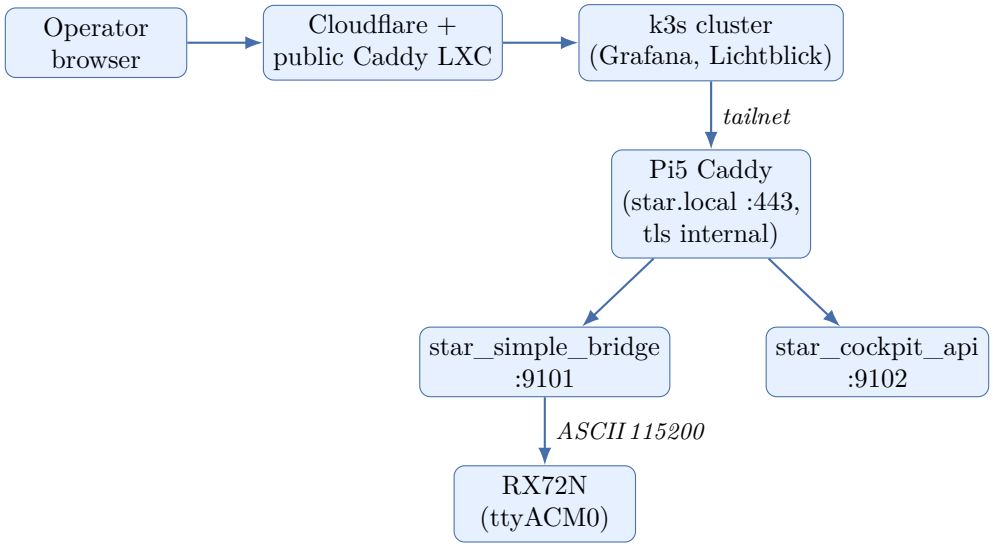

Figure 1: Operator connection topology. The Pi5 appears to the operator as `star.local`; Grafana and Lichtblick live in a homelab k3s cluster that is also on the Tailscale mesh.

4 Installation and First-Time Setup

This section covers procedures you perform once per site or robot. For daily operation see Section 5.

4.1 Prerequisites

- STAR robot with charged battery pack, RPLiDAR C1, BNO055 IMU, and four DRV8263H-driven motors.
- Raspberry Pi 5 running Ubuntu 24.04 with ROS2 Jazzy installed per `docs/PI_DEPLOYMENT.md`.
- Operator laptop with a modern browser and a Tailscale client enrolled in the same tailnet as the Pi5.
- USB-C power for the workstation, Renesas E2 Lite debugger for initial firmware flashing, and a USB-A to micro-USB cable for SCI boot-mode fallback.

4.2 Provisioning the Pi5

Follow `docs/PI_DEPLOYMENT.md` end-to-end. At a minimum:

1. Install ROS2 Jazzy from the OSRF apt repository and source `/opt/ros/jazzy/setup.bash`.
2. Install the STAR udev rules for `/dev/star-mcu` and `/dev/star-lidar` so device paths are stable across reboots.
3. Enroll the Pi5 in Tailscale and confirm its address resolves to `100.64.0.6` (or whichever tailnet address is documented for your team).
4. Build the ROS2 workspace with `./build-ros2.sh`.

4.3 Installing Pi5 Infrastructure Files

Refer to `infra/README.md`. Copy the system files into place on a fresh Pi5:

```
# From the repository root on the Pi5
sudo cp infra/pi5/opt/star_cockpit_api/server.py \
        /opt/star_cockpit_api/
sudo chmod +x /opt/star_cockpit_api/server.py

sudo cp infra/pi5/etc/caddy/Caddyfile /etc/caddy/Caddyfile
sudo cp infra/pi5/etc/systemd/system/*.service \
        /etc/systemd/system/

# Install Caddy (arm64)
sudo curl -fsSL -o /usr/local/bin/caddy \
  "https://caddyserver.com/api/download?os=linux&arch=arm64"
sudo chmod +x /usr/local/bin/caddy

sudo systemctl daemon-reload
sudo systemctl enable --now caddy.service
sudo systemctl enable --now star-cockpit-api.service
sudo systemctl enable --now star-slam-mvp.service
```

After this, rebooting the Pi5 brings up the complete operator stack automatically: Caddy on port 443, cockpit API on 9102, `star_simple_bridge` on 9101, and the SLAM MVP ROS2 node set in MANUAL mode with the emergency stop cleared.

4.4 Flashing the RX72N Firmware

Two flashing methods are supported by `scripts/flash-rx72n.sh`:

E2 Lite (default, recommended for lab work):

- SW4 Pin1 = OFF, Pin2 = OFF (Single Chip Mode).
- Board powered externally.
- FINE interface at 250 kbit/s.
- 1 Mbit/s FINE causes framing errors on the STAR PCB and must not be used.

SCI Boot Mode (field fallback via on-board USB-UART):

- SW4 Pin1 = ON, Pin2 = OFF.
- Power-cycle the board AFTER setting the switches.
- After a successful flash, return SW4 Pin1 = OFF and power-cycle again to run the new image.

On Apple Silicon and ARM Linux workstations, Renesas only ships `rfp-cli` as an x86_64 binary. The `flash-rx72n.sh` helper detects an aarch64 host and launches `rfp-cli` through `box64` when present, which avoids a known SIGSEGV that the kernel's binfmt + qemu-user fallback hits during `rfp-cli` startup. Install `box64` once on aarch64 hosts; x86_64 hosts run `rfp-cli` natively.

```
# Build a release image and flash via E2 Lite
make build-rx72n-release
./scripts/flash-rx72n.sh \
    star-rx72n-firmware/build/release/star-rx72n.mot

# Fallback over on-board USB-UART (SCI Boot Mode)
./scripts/flash-rx72n.sh \
    star-rx72n-firmware/build/release/star-rx72n.mot \
    sci /dev/ttyACM0
```

4.5 Grafana and Lichtblick Deployment

Grafana, Prometheus, and Lichtblick are deployed as k3s workloads on the homelab cluster (Tailscale address `100.64.0.1`). The Grafana dashboard is pushed into running Grafana via its HTTP API, so the authoritative source of truth is the JSON file at `monitoring/grafana-star-dashboard.json`. After any edit, re-pushing the dashboard is a one-step operation the cluster admin performs out of band.

The Grafana dashboard iframes the STAR map served by `star_simple_bridge` at `https://star.local/map.png`, which requires the operator's browser to trust the Pi5 Caddy's local CA. See `docs/ARCHITECTURE.md` for the one-time browser trust-store step.

4.6 Optional: Local Lichtblick Web Bundle

The public Lichtblick URL is the primary 3D viz path. For demos in places without reliable internet (the senior-design showcase floor in particular), the script `scripts/setup-lichtblick-web.sh` downloads the `lichtblick-suite/lichtblick` static web bundle to `/opt/star-lichtblick-web` so the Pi5 can serve it locally over the on-board AP at `http://192.168.50.1:8081`. The Pi5

default layout at `star-ros2/src/star_bringup/foxglove/star-default.json` is preserved on every run. [IMPLEMENTED].

```
# One-time, with the Pi5 on the internet
sudo ./scripts/setup-lichtblick-web.sh
# Default install pins lichtblick v1.24.3.
```

4.7 Bring-Up Helper: `slam-mvp.sh`

`scripts/slam-mvp.sh` is the canonical one-command bring-up that `star-slam-mvp.service` runs on boot. It is also useful on a developer workstation. The four subcommands:

- `slam-mvp.sh start [-mcu /dev/star-mcu] [-lidar /dev/star-lidar]` – launches each ROS2 node in the MVP set as a separate background process with its own log file under `/tmp/slam-mvp-logs/`. PIDs land in `/tmp/slam-mvp.pids`.
- `slam-mvp.sh status` – reports which PIDs are still alive and which logs were written.
- `slam-mvp.sh stop` – sends SIGTERM, then SIGKILL after a grace window.
- `slam-mvp.sh logs <node>` – tails the per-node log file (e.g. `slam-mvp.sh logs simple_bridge`).

If `/dev/star-mcu` is missing at start time – which happens when the E2 Lite holds the RX72N in reset during power-up ordering and starves the Cypress USB-UART – the script automatically cycles the E2 Lite USB device (vendor `045b`, product `82a0`) to release reset. The MCU enumerates within roughly one second.

4.8 Verifying the Install

From the operator's browser on the tailnet, confirm each surface is reachable:

- `https://star.local/` – Pi5 Caddy serves `star_simple_bridge`; `/map.png` returns an image (or a 404 until SLAM has produced a map).
- `https://star.local/api/state` – cockpit API returns `{autonomy, estop, speed, slam}` JSON.
- Grafana URL (deployed through public Caddy + Cloudflare) – the STAR dashboard loads and panels populate within seconds.
- Lichtblick URL (deployed through public Caddy + Cloudflare) – the default layout loads; click "Open Connection" and enter `ws://100.64.0.6:8765` to attach to the foxglove bridge on the robot.

5 Daily Operation Procedures

5.1 Power-On Sequence

1. Verify the battery pack is seated and the main breaker is OFF.
2. Flip the main breaker ON. The Pi5 boots; the RX72N status LED begins its breathing pattern within a second.
3. Wait approximately 30 seconds for ROS2, Caddy, and the cockpit API to come up (driven by `star-slam-mvp.service`).

4. Confirm Tailscale has assigned the Pi5 its tailnet address. (The Pi5 side has a 8 s USB-enumeration delay before the SLAM MVP script starts.)

5. The robot is now in MANUAL mode with the emergency stop cleared. It will not move until you command it.

5.2 Opening the Cockpit

Open two browser tabs from the operator laptop:

1. The public Grafana URL (see Section 4). The STAR dashboard has control panels for autonomy, e-stop, speed, and SLAM, plus live telemetry gauges.

2. The public Lichtblick URL. The default layout loads the 3D scene, LiDAR, map, stereo point cloud, and IMU plots. In "Open Connection", enter `ws://100.64.0.6:8765`.

5.3 Granting the Browser Trust for `star.local`

The Grafana dashboard loads map images directly from `https://star.local/map.png`. This URL is served by the Pi5's Caddy using a local CA. Install that CA on the operator laptop one time (per `docs/ARCHITECTURE.md`), otherwise the map panel will show a broken-image placeholder.

5.4 Teleoperation (MANUAL mode)

In MANUAL mode the robot follows velocity commands from the Grafana cockpit or from a publisher on `/cmd_vel`. Two input options:

- **Grafana speed panel:** POST to `/api/speed` with a scalar in $[0, 1]$, then POST linear/angular velocity deltas to `/api/cmd_vel`.

- **USB gamepad** (at the workstation): left stick Y axis commands linear velocity (inverted), left stick X axis commands angular velocity; deadzone is 0.1; commands publish at 50 Hz.

The `star_safety_monitor` enforces a linear ceiling of 1.0 m/s and an angular ceiling of 2.0 rad/s. Commands outside this envelope are clamped.

5.5 Switching to AUTONOMY

From the Grafana cockpit, click the AUTONOMY toggle. This sends `POST /api/autonomy/toggle` to the cockpit API, which flips the latched `/star/autonomy_enable` topic. When AUTONOMY is on, the supervisor forwards Nav2's `/nav2/cmd_vel` to the gated `/cmd_vel_out` that `star_simple_bridge` consumes; operator teleop is ignored.

To give a Nav2 goal, click a target on the Lichtblick 3D panel configured to publish to `/goal_pose`. Nav2 plans and executes, publishing status back to the topic tree.

5.6 Running SLAM

If SLAM is not already active:

1. Click the SLAM START toggle in Grafana (POSTs to `/api/slam/start`). The cockpit API activates the `/slam_toolbox` lifecycle node.

2. Drive the robot slowly through the space. The Lichtblick map panel fills in as scans accumulate. A separate SLAM map can be viewed directly at `https://star.local/map.png`.

3. When finished, click SLAM STOP to deactivate the node. To reset the map, send `POST /map/reset?confirm=yes` to `https://star.local/map/reset` on `star_simple_bridge`.

5.7 Running a Compliance Audit

Compliance nodes (Section 7) publish check results on the `/compliance/*` topics whenever SLAM is active and the robot is moving. The ramp-slope check is [IMPLEMENTED]; other checks are [STRETCH] or [ARCHITECTED] per the SDD traceability matrix. The report generator in `final_capstone/compliance-engine/star_compliance/report/` produces a PDF audit report at the end of a run; copy it off the Pi5 over SCP.

5.8 Graceful Shutdown

1. From Grafana, clear AUTONOMY (back to MANUAL) and optionally SLAM STOP.
2. SSH to the Pi5 and run `sudo systemctl stop star-slam-mvp.service` to bring down ROS2 nodes gracefully.
3. Run `sudo shutdown -h now` on the Pi5.
4. Wait for the Pi5 power LED to extinguish, then flip the main breaker OFF.

6 Dashboard Reference

6.1 Grafana Cockpit

The committed dashboard is `monitoring/grafana-star-dashboard.json`. It contains row groups for SLAM map, motors, safety, sensors, and SLAM diagnostics. Key widgets the operator uses:

- **Map panel:** iframe-style HTML rendering of `https://star.local/map.png`. Auto-refreshes on the Grafana time-range tick.
- **Motor gauges:** per-wheel velocity in m/s (metric `star_motor_velocity_mps{wheel=...}`).
- **E-stop button and indicator:** POST `/api/estop/toggle`. The indicator gauge reflects the latched `/star/estop` topic.
- **Autonomy toggle:** POST `/api/autonomy/toggle`.
- **Speed slider:** POST `/api/speed` with a scalar in $[0, 1]$ that becomes `/star/speed_scale`.
- **SLAM start/stop buttons:** POST `/api/slam/start` or `/api/slam/stop`.

Grafana's minimum refresh interval is configured to 1 s via the `GF_DASHBOARDS_MIN_REFRESH_INTERVAL` environment variable on the Grafana pod.

6.2 Lichtblick Layout

The default layout `star-ros2/src/star_bringup/foxglove/star-default.json` wires the following topics into Lichtblick's 3D panel:

- `/scan` – RPLiDAR C1 scan (visible, green, 5 px).
- `/map` – occupancy grid from `slam_toolbox`.
- `/cloud_map` – 3D point cloud, height-colored, 6 px.

- `/octomap_occupied_space` – Octomap (hidden by default; enable for obstacle review).
- `/stereo/points2` – stereo-camera point cloud, 4 px, rainbow-colored.
- `/tf` – transform tree.
- `/odometry/filtered` – EKF odometry, arrow marker.
- `/odom/unfiltered` – raw wheel odometry, arrow marker.

Foxglove Gauge Mirror Topics

`sensor_msgs/Imu` and the wheel `Float32MultiArray` topics carry nested fields that Foxglove Gauge panels cannot read directly. `star_simple_bridge` mirrors the relevant scalars into single-valued `std_msgs/Float32` topics so each Gauge binds to one topic. These are the topics to use when building custom Foxglove panels:

Table 2: Scalar telemetry topics for Foxglove Gauge panels.

Topic	Meaning
`/motor/fl/velocity_mps`, `/motor/fr/...`	Per-wheel signed linear velocity in m/s. Repeated for `bl` and `br`.
`/motor/fl/duty_percent`, `/motor/fr/...`	Per-wheel signed PWM duty in percent (−100 to +100), including the bl and br wheels.
`/imu/roll_deg`, `/imu/pitch_deg`, `/imu/heading_deg`	BNO055 fused orientation in degrees.
`/imu/gyro_x_dps`, `/imu/gyro_y_dps`, `/imu/gyro_z_dps`	Body-frame angular velocity in degrees per second.
`/imu/accel_x_mps2`, `/imu/accel_y_mps2`, `/imu/accel_z_mps2`	Gravity-compensated linear acceleration in m/s^2.

The same scalars are simultaneously exposed as Prometheus gauges on `:9100` (e.g. `star_motor_velocity_mps{whe` `star_imu_orientation_deg{axis="heading"}`) so a Grafana Gauge panel binds the same datum without going through ROS2.

6.3 Foxglove Studio (desktop alternative)

Foxglove Studio can load the same layout JSON. Connect to `ws://100.64.0.6:8765` (the foxglove bridge on the Pi5). This is useful when the public Lichtblick URL is unavailable or for power users who prefer the desktop app.

7 Topic, Endpoint, and Port Reference

7.1 Ports Exposed on the Pi5

Table 3: Network ports exposed on the Pi5.

Port	Service	Role
443	Caddy (tls internal)	Fronts `star.local`
8765	foxglove_bridge	ROS2 over WebSocket
9100	node_exporter	Scraped by Alloy / Prometheus
9101	star_simple_bridge	`/map.*`, metrics, map reset
9102	star_cockpit_api	Cockpit HTTP API

7.2 Cockpit HTTP API

The following endpoints are exposed by `star_cockpit_api` on `100.64.0.6:9102` (and are reverse-proxied under `https://star.local/api/*` via Caddy).

Table 4: `star_cockpit_api` endpoints.

Method	Path	Purpose
GET	`/api/state`	Cached `{autonomy, estop, speed, slam}`
POST	`/api/autonomy/toggle`	Flip `/star/autonomy_enable`
POST	`/api/estop/toggle`	Flip `/star/estop`
POST	`/api/speed`	Body `{"value":0.75}` -> `/star/speed_scale`
POST	`/api/cmd_vel`	Body `{"linear":0.5,"angular":0}` -> `/cmd_vel`
POST	`/api/slam/start`	Configure + activate `/slam_toolbox`
POST	`/api/slam/stop`	Deactivate `/slam_toolbox`

7.3 ROS2 Topics the Operator Cares About

Table 5: Operator-relevant ROS2 topics.

Topic	Type	Notes
`/cmd_vel`	Twist	Manual teleop input
`/nav2/cmd_vel`	Twist	Nav2 output (AUTONOMY)
`/cmd_vel_out`	Twist	Gated output to bridge
`/goal_pose`	PoseStamped	Click target in Lichtblick
`/star/autonomy_enable`	Bool (latched)	MANUAL vs AUTONOMY
`/star/estop`	Bool (latched)	Soft e-stop
`/star/speed_scale`	Float32 $[0, 1]$	Speed governor
`/odom/unfiltered`	Odometry	Raw wheel odometry
`/odometry/filtered`	Odometry	EKF output
`/imu/data`	Imu	Fused BNO055 data
`/scan`	LaserScan	RPLiDAR C1
`/map`	OccupancyGrid	SLAM output
`/compliance/report`	Custom	Aggregate ADA result
`/compliance/ramp_slope`	Custom	Per-check result

8 Safety, Warnings, and Emergency Procedures

8.1 Software Emergency Stop

Three independent paths can stop the robot:

1. Click the E-STOP button on the Grafana dashboard (POSTs to `/api/estop/toggle`). This latches `/star/estop` to true and the supervisor gates `/cmd_vel_out` to zero.
2. Publish `true` directly on `/star/estop` (for example, from a command-line `ros2 topic pub` in an SSH session).
3. Physically disconnect USB to `/dev/star-mcu`; the `star_simple_bridge` stops delivering velocity commands and the RX72N watchdog zeroes the motor outputs within one control period.

8.2 Auto-E-Stop Thresholds

The `star_safety_monitor` ROS2 node enforces the limits in Table 6. Values come from `star-ros2/src/star_saf` and are authoritative.

Table 6: Safety-monitor thresholds.

Parameter	Value	Behavior
`heartbeat_timeout_ms`	500	Alarm if no heartbeat
`max_linear_velocity`	1.0	Clamped to 1 m/s
`max_angular_velocity`	2.0	Clamped to 2 rad/s
`stall_detection_threshold`	0.05	Below this counts as stall
`stall_samples_required`	5	Consecutive stalls -> E-stop
`publish_rate`	10.0	Diagnostics at 10 Hz
`obstacle_estop_distance`	0.10	Sonar < 10 cm -> E-stop
`obstacle_warn_distance`	0.30	Sonar < 30 cm -> warn
`obstacle_clear_count_required`	10	10 clear samples -> release
`enable_auto_estop`	true	Master enable
`estop_recovery_delay`	5.0	Delay before auto-recovery

8.3 Operator Safety Rules

- Never reach under the robot while the main breaker is ON.
- Keep the emergency-stop browser tab visible at all times during AUTONOMY.
- Do not lift the robot by the LiDAR dome or by any wire harness. Use the chassis mounts only.
- Before flashing firmware, set the robot on blocks so the wheels are off the ground.
- Never insert SCI-boot mode (SW4 Pin1 = ON) without a deliberate plan to reset the switches afterwards; the robot cannot run user firmware in that state.

9 Troubleshooting and Error Codes

9.1 Common Failure Modes

Problem: The Grafana cockpit map panel shows a broken image. The browser does not trust the Pi5 Caddy's local CA. Install the CA per `docs/ARCHITECTURE.md` and reload. If

the problem persists, confirm `curl -k https://star.local/map.png` from any machine on the tailnet returns an image.

Problem: Grafana panels are empty. Prometheus is not receiving metrics. On the Pi5, run `systemctl status alloy` and confirm the Alloy agent is exporting to `http://100.64.0.1:30909/api/v1/write`. Restart with `sudo systemctl restart alloy`.

Problem: Lichtblick cannot connect to `ws://100.64.0.6:8765`. The foxglove bridge did not start. SSH to the Pi5 and check `systemctl status star-slam-mvp.service` and the log at `/tmp/slam-mvp-logs/`. Restart with `sudo systemctl restart star-slam-mvp.service`.

Problem: AUTONOMY toggle has no effect. The supervisor is not subscribed, or the cockpit API is down. `curl https://star.local/api/state` should return valid JSON. If not, restart with `sudo systemctl restart star-cockpit-api.service`.

Problem: Robot stalls and emergency stop engages. The safety monitor detected a wheel velocity below `stall_detection_threshold` for five consecutive samples. Check for mechanical binding, low battery, or a disconnected motor wire. After clearing the cause, wait `estop_recovery_delay` seconds and the monitor re-enables motion.

Problem: Firmware flash fails on E2 Lite. Confirm SW4 Pin1 = OFF and Pin2 = OFF. Confirm the board has external power (E2 Lite does not power the STAR PCB). Confirm FINE speed is 250 kbit/s; 1 Mbit/s is known to cause framing errors on our PCB.

9.2 RX72N Error Code Ranges

When a firmware error surfaces on a ROS2 diagnostic message, the `rx_err_t` code maps to a range:

- `0x000` – success.
- `0x100-0x1FF` – generic error (null pointer, invalid argument, not initialized). Restart the bridge or power-cycle.
- `0x200-0x2FF` – hardware error (SPI, I2C, GPIO). Check wiring and peripheral config.
- `0x300-0x3FF` – RTOS error (ThreadX semaphore, mutex, queue). Reboot the RX72N.
- `0x400-0x4FF` – communication error (CRC, framing, HARQ exhausted). Inspect SPI connector; reflash firmware if persistent.
- `0x500-0x5FF` – validation error (out-of-range input, boundary check failed). Software-side bug; file an issue against the repo.

9.3 LED Status Patterns

The on-board LEDs managed by `star-rx72n-firmware/blinky/main.c` breathe sequentially across the six indicators as an "alive" heartbeat. A static (non- breathing) pattern indicates the RX72N firmware has halted; power- cycle the board and observe the boot LED sequence.

10 Maintenance

10.1 Firmware Update

1. On the workstation, `git pull` the repository.

2. Rebuild: `make build-rx72n-release`.
3. Set the robot on blocks so the wheels are free.
4. Flash via E2 Lite: `./scripts/flash-rx72n.sh star-rx72n-firmware/build/release/star-rx72n.mot`.
5. Observe the LED sequence and confirm heartbeat.
6. Power-cycle the board and verify the robot responds to a manual cmd_vel pulse in the Grafana dashboard.

10.2 Rebuilding the ROS2 Workspace

```
cd /workspaces/STAR    # or wherever the repo lives on the Pi5
./build-ros2.sh        # wraps colcon build with Cyclone DDS
sudo systemctl restart star-slam-mvp.service
```

10.3 Bring-Up Tests

Each hardware subsystem has a standalone smoke test under `star-rx72n-firmware/`:

- `motor_spin_test/` – spin each wheel open-loop.
- `pwm_test/` – scope the PWM waveform on the motor connectors.
- `encoder_test/` – print quadrature counts at runtime.
- `gpio_test/` – toggle every GPIO and verify with the external harness.
- `imu_test/` – print BNO055 orientation over UART.
- `uart_test/` – loopback test on the debug UART.
- `sonar_test/` – trigger and read HC-SR04 channels.

Each directory contains a CMake project; build and flash it in place of the main firmware, observe the expected behavior, then reflash the production image.

10.4 Updating the Dashboards

Grafana dashboard edits are stored back to `monitoring/grafana-star-dashboard.json`, committed, and pushed through the cluster admin's Grafana API import flow. Lichtblick layouts are distributed through the k3s ConfigMap-injected default layout; to edit, change `star-ros2/src/star_bringup/foxglo` commit, rebuild the Lichtblick deployment image, and re-apply.

10.5 Calibration

Three calibration constants live inside `star-ros2/src/star_simple_bridge/star_simple_bridge/simple_bridge_node.py` and must be revisited whenever the chassis hardware changes (motor swap, encoder swap, IMU re-mount).

Encoder ticks per wheel revolution (`TICKS_PER_REV`). The firmware reports raw quadrature counts at the motor shaft (341 PPR Hall sensor times 4x quadrature decoding = 1364 ticks per shaft revolution). Empirically, on the current goBILDA Wasteland chassis with two damaged motors, the geometric mean of two ruler-measured drives gave 1283, which is the constant currently checked in

(the script's comment block at the constant records the date and the per-drive values). Recalibrate when motors are replaced:

1. Mark a straight-line course of 1.00 m on the floor.
2. With `TICKS_PER_REV` at its current value, drive the robot the marked distance under teleop and read the integrated `/odom/unfiltered` pose-x.
3. If odom reads d_{odom} for an actual distance d_{ruler}, the new `TICKS_PER_REV` is `old` $\times\, d_{odom}/d_{ruler}$.
4. Average two drives in opposite directions to absorb any asymmetric wheel slip; commit the geometric mean.

SLAM toolbox's scan matching tolerates ±10 % drift, so the calibration only needs to be approximately right.

IMU mount flip (`IMU_MOUNT_FLIP_Y`). The BNO055 chip is soldered face-down on the PCB, so its native $+Z$ points into the board (downward in the robot frame). The bridge applies a 180-degree rotation about the IMU body Y axis before publishing `/imu/data`, so the published pitch is near 0 when the robot sits level. If the IMU is ever remounted right-side-up, set `IMU_MOUNT_FLIP_Y = False` and restart the bridge.

Encoder direction signs. On the current chassis the right-side quadrature A/B phase is swapped relative to the left, so the bridge negates d_{ticks} for the FR and BR wheels at the integration step. If a future harness rewire reverses one side's connector, flip the corresponding sign in `_handle_encoder_line`; verify with a +0.5 m/s forward command for 2 s producing positive `pose_x` of approximately 1.0 m.

Motor saturation and stiction floor. Two safety constants in the same module shape the per-wheel command:

- `MAX_WHEEL_MPS = 1.0` – when a combined linear+angular twist would push one wheel past 1 m/s, both wheels are scaled proportionally so the commanded turn ratio is preserved.
- `MIN_WHEEL_MPS_STICTION = 0.30` – a stiction-break floor that boosts each non-zero wheel command up to the stall threshold while preserving the sign. Required because the damaged motors stall below approximately 0.30 m/s commanded; remove this once motors are replaced and the Nav2 controller reaches 10 Hz loop rate.

10.6 Updating Firmware PID Gains

PID gain design is performed offline in MATLAB per the SDD. New gains are written to `star-rx72n-firmware/...` picked up by the firmware build, and deployed via the firmware-update procedure. There is no online autotuning path.

11 Assumptions and Limitations

Operator-facing subset of the SDD limitations:

- Four motors maximum; firmware is sized to exactly four GPTW and four TPU channels.
- Single operator at a time; the Grafana cockpit has no role separation or contention arbitration.

- Tailnet required; the cockpit is not reachable outside the Tailscale mesh and the public Cloudflare entry point.
- English only; no internationalization.
- Online PID autotuning is not supported. Plant or load changes require an offline MATLAB re-run and a firmware rebuild.
- Compliance coverage: only the ramp-slope check is [IMPLEMENTED]; path width and trip hazard are [STRETCH], and ramp width, ramp landings, door clear width, and door threshold are [ARCHITECTED] per the SDD traceability matrix.
- `star-simple` (a planned lightweight bring-up tree on the field Pi5) is [ARCHITECTED]: some of it is present on the physical robot but has not been committed back to the repository yet. Use `star_simple_bridge` directly for now.
- Grafana dashboards are deployed by the cluster admin through a Grafana HTTP API import; the operator cannot rebuild the dashboard without cluster access.

12 Glossary

ADA 405.2
: Section 405.2 of the 2010 ADA Standards, which limits ramp slope to 1:12 rise-over-run.

Caddy
: HTTP reverse proxy used in two places in STAR: on the Pi5 with `tls internal` (local CA) to front `star_simple_bridge` and `star_cockpit_api`; and on a separate homelab LXC as the public entry point behind Cloudflare.

Foxglove Studio
: Desktop ROS2 visualization application. STAR ships a Foxglove- compatible JSON layout.

Lichtblick
: Open-source web fork of Foxglove Studio. STAR's primary operator-facing 3D visualization, deployed on k3s.

Grafana
: Browser-based telemetry dashboard. STAR's primary cockpit.

Tailscale
: WireGuard-based VPN mesh. STAR's Pi5 lives at `100.64.0.6`; the homelab cluster at `100.64.0.1`.

k3s
: Lightweight Kubernetes distribution used to host Grafana, Prometheus, and Lichtblick in STAR's homelab cluster.

Alloy
: Grafana Labs telemetry agent installed on the Pi5 that remote- writes metrics to Prometheus.

SLAM MVP
: Minimal SLAM stack launched by `star-slam-mvp.service` at boot. Uses `slam_toolbox` in async mode.

Supervisor
: ROS2 node that gates `/cmd_vel` and `/nav2/cmd_vel` into `/cmd_vel_out` based on latched

autonomy and e-stop topics.

E2 Lite
Renesas JTAG / FINE debug probe used to program the RX72N.

Appendices

Appendix A: Quick Reference Card

Power on: Flip breaker -> wait 30 seconds for services -> open Grafana and Lichtblick tabs.

Emergency stop: Click E-STOP in Grafana, or `curl -X POST https://star.local/api/estop/toggle`.

MANUAL mode drive: Grafana speed slider + cmd_vel POST, or USB gamepad at the workstation.

AUTONOMY mode: Click AUTONOMY toggle in Grafana, then click a goal in Lichtblick.

SLAM start: Click SLAM START in Grafana; drive slowly; watch `/map.png` fill in.

Flash firmware: `./scripts/flash-rx72n.sh star-rx72n.mot` (E2 Lite default).

Shutdown: Clear AUTONOMY -> stop `star-slam-mvp.service` -> `sudo shutdown -h now` -> flip breaker OFF.

Appendix B: Config File Cheat Sheet

File	Purpose
`star-ros2/src/star_bringup/config/slam_toolbox.yaml`	SLAM tuning (resolution, loop closure)
`star-ros2/src/star_bringup/config/nav2_params.yaml`	Nav2 planner + controller parameters
`star-ros2/src/star_bringup/config/ekf.yaml`	robot_localization EKF parameters
`star-ros2/src/star_bringup/config/explore_params.yaml`	`explore_lite` frontier search parameters
`star-ros2/src/star_safety_monitor/config/safety_monitor.yaml`	Safety-monitor thresholds (Table 6)
`infra/pi5/etc/caddy/Caddyfile`	Pi5-side Caddy (`star.local`, tls internal)
`monitoring/grafana-star-dashboard.json`	Grafana dashboard definition
`star-ros2/src/star_bringup/foxglove/star-default.json`	Lichtblick / Foxglove layout

Appendix C: Important Addresses and Credentials

- Pi5 tailnet address: `100.64.0.6` (hostname `star.local`).
- Homelab cluster tailnet address: `100.64.0.1` (Prometheus remote-write `http://100.64.0.1:30909/api/v1/w`
- Pi5 serial device for RX72N: `/dev/star-mcu` (udev-aliased to `/dev/ttyACM0`).
- Pi5 serial device for RPLiDAR C1: `/dev/star-lidar`.
- Local-CA trust: install the Pi5 Caddy local CA on every operator laptop (one-time, per `docs/ARCHITECTURE.md`).

- Public URLs for Grafana and Lichtblick are deployed via Cloudflare and the homelab public Caddy; consult the cluster admin for the current host names.

Appendix D: Change Log

- Version 1.0, April 28, 2026 – Initial release for ESET 420 capstone submission.